

---

# ***m*HC: Manifold-Constrained Hyper-Connections**

---

Zhenda Xie<sup>\*1</sup> Yixuan Wei<sup>\*1</sup> Huanqi Cao<sup>\*1</sup> Chenggang Zhao<sup>1</sup> Chengqi Deng<sup>1</sup> Jiashi Li<sup>1</sup> Damai Dai<sup>1</sup>  
Huazuo Gao<sup>1</sup> Mingyu Xu<sup>1</sup> Kuai Yu<sup>1</sup> Liang Zhao<sup>1</sup> Shangyan Zhou<sup>1</sup> Zhean Xu<sup>1</sup> Zhengyan Zhang<sup>1</sup>  
Wangding Zeng<sup>1</sup> Shengding Hu<sup>1</sup> Yuqing Wang<sup>1</sup> Jingyang Yuan<sup>1,2</sup> Lean Wang<sup>1,2</sup> Wenfeng Liang<sup>1</sup>

## **Abstract**

Recently, studies exemplified by Hyper-Connections (HC) have extended the ubiquitous residual connection paradigm established over the past decade by expanding the residual stream width and diversifying connectivity patterns. While yielding substantial performance gains, this diversification fundamentally compromises the identity mapping property intrinsic to the residual connection, which causes severe training instability and restricted scalability, and additionally incurs notable memory access overhead. To address these challenges, we propose **Manifold-Constrained Hyper-Connections (*m*HC)**, a general framework that projects the residual connection space of HC onto a specific manifold to restore the identity mapping property, while incorporating rigorous infrastructure optimization to ensure efficiency. Empirical experiments demonstrate that *m*HC is effective for training at scale, offering tangible performance improvements and superior scalability. We anticipate that *m*HC, as a flexible and practical extension of HC, will contribute to a deeper understanding of topological architecture design and suggest promising directions for the evolution of foundational models.

## **1. Introduction**

Deep neural network architectures have undergone rapid evolution since the introduction of ResNets (He et al., 2016a). As illustrated in Fig. 1 (a), the structure of a single layer can be formulated as follows:

$$\mathbf{x}_{l+1} = \mathbf{x}_l + \mathcal{F}(\mathbf{x}_l, \mathcal{W}_l), \quad (1)$$

---

<sup>\*</sup>Equal contribution <sup>1</sup>DeepSeek-AI <sup>2</sup>School of Computer Science, Peking University. Correspondence to: Zhenda Xie <xie.zhenda@deepseek.com>.

where  $\mathbf{x}_l$  and  $\mathbf{x}_{l+1}$  denote the  $C$ -dimensional input and output of the  $l$ -th layer, respectively, and  $\mathcal{F}$  represents the residual function. Although the residual function  $\mathcal{F}$  has evolved over the past decade to include various operations such as convolution, attention mechanisms, and feed-forward networks, the paradigm of the residual connection has maintained its original form. Accompanying the progression of Transformer (Vaswani et al., 2017) architecture, this paradigm has currently established itself as a fundamental design element in large language models (LLMs) (Brown et al., 2020; Touvron et al., 2023; Liu et al., 2024b).

This success is primarily attributed to the concise form of the residual connection. More importantly, early research (He et al., 2016b) revealed that the identity mapping property of the residual connection maintains stability and efficiency during large-scale training. By recursively extending the residual connection across multiple layers, Eq. (1) yields:

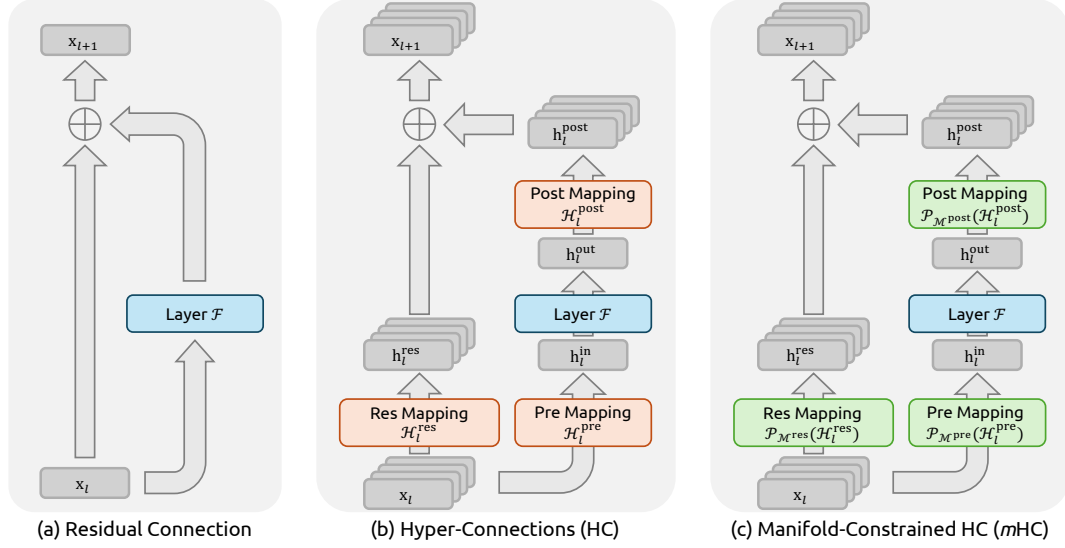
$$\mathbf{x}_L = \mathbf{x}_l + \sum_{i=l}^{L-1} \mathcal{F}(\mathbf{x}_i, \mathcal{W}_i), \quad (2)$$

where  $L$  and  $l$  correspond to deeper and shallower layers, respectively. The term identity mapping refers to the component  $\mathbf{x}_l$  itself, which emphasizes the property that the signal from the shallower layer maps directly to the deeper layer without any modification.

Recently, studies exemplified by Hyper-Connections (HC) (Zhu et al., 2025) have introduced a new dimension to the residual connection and empirically demonstrated its performance potential. The single-layer architecture of HC is illustrated in Fig. 1 (b). By expanding the width of the residual stream and enhancing connection complexity, HC significantly increases topological complexity without altering the computational overhead of individual units regarding FLOPs. Formally, single-layer propagation in HC is defined as:

$$\mathbf{x}_{l+1} = \mathcal{H}_l^{\text{res}} \mathbf{x}_l + \mathcal{H}_l^{\text{post}^\top} \mathcal{F}(\mathcal{H}_l^{\text{pre}} \mathbf{x}_l, \mathcal{W}_l), \quad (3)$$

where  $\mathbf{x}_l$  and  $\mathbf{x}_{l+1}$  denote the input and output of the  $l$ -th layer, respectively. Unlike the formulation in Eq. (1), the feature dimension of  $\mathbf{x}_l$  and  $\mathbf{x}_{l+1}$  is expanded from  $C$  to  $n \times C$ , where  $n$  is the expansion rate. The term  $\mathcal{H}_l^{\text{res}} \in \mathbb{R}^{n \times n}$  represents a learnable mapping that mixes features



**Figure 1. Illustrations of Residual Connection Paradigms.** This figure compares the structural design of (a) standard Residual Connection, (b) Hyper-Connections (HC), and (c) our proposed **Manifold-Constrained Hyper-Connections (mHC)**. Unlike the unconstrained HC, mHC focuses on optimizing the residual connection space by projecting the matrices onto a constrained manifold to ensure stability.

within the residual stream. Also as a learnable mapping,  $\mathcal{H}_i^{\text{pre}} \in \mathbb{R}^{1 \times n}$  aggregates features from the  $nC$ -dim stream into a  $C$ -dim layer input, and conversely,  $\mathcal{H}_i^{\text{post}} \in \mathbb{R}^{1 \times n}$  maps the layer output back onto the stream.

However, as the training scale increases, HC introduces potential risks of instability. The primary concern is that the unconstrained nature of HC compromises the identity mapping property when the architecture extends across multiple layers. In architectures comprising multiple parallel streams, an ideal identity mapping serves as a conservation mechanism. It ensures that the average signal intensity across streams remains invariant during both forward and backward propagation. Recursively extending HC to multiple layers via Eq. (3) yields:

$$\begin{aligned} \mathbf{x}_L = & \left( \prod_{i=1}^{L-l} \mathcal{H}_{L-i}^{\text{res}} \right) \mathbf{x}_l \\ & + \sum_{i=l}^{L-1} \left( \prod_{j=1}^{L-1-i} \mathcal{H}_{L-j}^{\text{res}} \right) \mathcal{H}_i^{\text{post} \top} \mathcal{F}(\mathcal{H}_i^{\text{pre}} \mathbf{x}_i, \mathcal{W}_i), \end{aligned} \quad (4)$$

where  $L$  and  $l$  represent a deeper layer and a shallower layer, respectively. In contrast to Eq. (2), the composite mapping  $\prod_{i=1}^{L-l} \mathcal{H}_{L-i}^{\text{res}}$  in HC fails to preserve the global mean of the features. This discrepancy leads to unbounded signal amplification or attenuation, resulting in instability during large-scale training. A further consideration is that, while HC preserves computational efficiency in terms of FLOPs, the hardware efficiency concerning memory access

costs for the widened residual stream remains unaddressed in the original design, as it requires frequently reading and writing the full residual stream, which is carefully analyzed in Appendix A.1. These factors collectively restrict the practical scalability of HC and hinder its application in large-scale training.

To address these challenges, we propose **Manifold-Constrained Hyper-Connections (mHC)**, as shown in Fig. 1(c), a general framework that projects the residual connection space of HC onto a specific manifold to restore the identity mapping property, while incorporating rigorous infrastructure optimization to ensure efficiency. Specifically, mHC utilizes the Sinkhorn-Knopp algorithm (Sinkhorn & Knopp, 1967) to entropically project  $\mathcal{H}_i^{\text{res}}$  onto the Birkhoff polytope. This operation effectively constrains the residual connection matrices within the manifold that is constituted by doubly stochastic matrices. Since the row and column sums of these matrices equal 1, the operation  $\mathcal{H}_i^{\text{res}} \mathbf{x}_l$  functions as a convex combination of the input features. This characteristic facilitates a well-conditioned signal propagation where the feature mean is conserved, and the signal norm is strictly regularized, effectively mitigating the risk of vanishing or exploding signals. Furthermore, due to the closure of matrix multiplication for doubly stochastic matrices, the composite mapping  $\prod_{i=1}^{L-l} \mathcal{H}_{L-i}^{\text{res}}$  retains this conservation property. Consequently, mHC effectively maintains the stability of identity mappings between arbitrary depths. To ensure efficiency, we employ kernel fusion and develop mixed precision kernels utilizing TileLang (Wang et al., 2025). Furthermore, we mitigate the memory foot-

print through selective recomputing and carefully overlap communication within the DualPipe schedule (Liu et al., 2024b).

Extensive experiments on language model pretraining demonstrate that *m*HC exhibits exceptional stability and scalability while maintaining the performance advantages of HC. Large-scale training indicates that *m*HC supports training at scale and introduces only a 6.7% additional time overhead when expansion rate  $n = 4$ .

## 2. Related Works

Architectural advancements in deep learning can be primarily classified into *micro-design* and *macro-design*. Micro-design concerns the internal architecture of computational blocks, specifying how features are processed across spatial, temporal, and channel dimensions. In contrast, macro-design establishes the inter-block topological structure, thereby dictating how feature representations are propagated, routed, and merged across distinct layers.

### 2.1. Micro Design

Driven by parameter sharing and translation invariance, convolution initially dominated the processing of structured signals. While subsequent variations such as depthwise separable (Chollet, 2017) and grouped convolutions (Xie et al., 2017) optimized efficiency, the advent of Transformers (Vaswani et al., 2017) established Attention and Feed-Forward Networks (FFNs) as the fundamental building blocks of modern architecture. Attention mechanisms facilitate global information propagation, while FFNs enhance the representational capacity of individual features. To balance performance with the computational demands of LLMs, attention mechanisms have evolved towards efficient variants such as Multi-Query Attention (MQA) (Shazeer, 2019), Grouped-Query Attention (GQA) (Ainslie et al., 2023), and Multi-Head Latent Attention (MLA) (Liu et al., 2024a). Simultaneously, FFNs have been generalized into sparse computing paradigms via Mixture-of-Experts (MoE) (Shazeer et al., 2017; Lepikhin et al., 2020; Fedus et al., 2022), allowing for massive parameter scaling without proportional computational costs.

### 2.2. Macro Design

Macro-design governs the global topology of the network (Srivastava et al., 2015). Following ResNet (He et al., 2016a), architectures such as DenseNet (Huang et al., 2017) and FractalNet (Larsson et al., 2016) aimed to enhance performance by increasing topological complexity through dense connectivity and multi-path structures, respectively. Deep Layer Aggregation (DLA) (Yu et al., 2018) further extended this paradigm by recursively aggregating features

across various depths and resolutions.

More recently, the focus of macro-design has shifted toward expanding the width of the residual stream (Chai et al., 2020; Fang et al., 2023; Xie et al., 2023; Pagliardini et al., 2024; Menghani et al., 2025; Heddes et al., 2025; Zhu et al., 2025; Mak & Flanigan, 2025; Xiao et al., 2025). Hyper-Connections (HC) (Zhu et al., 2025) introduced learnable matrices to modulate connection strengths among features at varying depths, while the Residual Matrix Transformer (RMT) (Mak & Flanigan, 2025) replaced the standard residual stream with an outer-product memory matrix to facilitate feature storage. Similarly, MUDDFormer (Xiao et al., 2025) employs multiway dynamic dense connections to optimize cross-layer information flow. Despite their potential, these approaches compromise the inherent identity mapping property of the residual connection, thereby introducing instability and hindering scalability. Furthermore, they incur significant memory access overhead due to expanded feature widths. Building upon HC, the proposed *m*HC restricts the residual connection space onto a specific manifold to restore the identity mapping property, while also incorporating rigorous infrastructure optimizations to ensure efficiency. This approach enhances stability and scalability while maintaining the topological benefits of expanded connections.

## 3. Preliminary

We first establish the notation used in this work. In the HC formulation, the input to the  $l$ -th layer,  $\mathbf{x}_l \in \mathbb{R}^{1 \times C}$ , is expanded by a factor of  $n$  to construct a hidden matrix  $\mathbf{x}_l = (\mathbf{x}_{l,0}^\top, \dots, \mathbf{x}_{l,n-1}^\top)^\top \in \mathbb{R}^{n \times C}$  which can be viewed as  $n$ -stream residual. This operation effectively broadens the width of the residual stream. To govern the read-out, write-in, and updating processes of this stream, HC introduces three learnable linear mappings— $\mathcal{H}_l^{\text{pre}}, \mathcal{H}_l^{\text{post}} \in \mathbb{R}^{1 \times n}$ , and  $\mathcal{H}_l^{\text{res}} \in \mathbb{R}^{n \times n}$ . These mappings modify the standard residual connection shown in Eq. (1), resulting in the formulation given in Eq. (3). For the first layer, the initial multi-stream input  $\mathbf{x}_0 \in \mathbb{R}^{n \times C}$  is constructed by replicating the token embedding  $n$  times along the stream dimension.

In the HC formulation, learnable mappings are composed of two parts of coefficients: the input-dependent one and the global one, referred to as dynamic mappings and static mappings, respectively. Formally, HC computes the coefficients as follows:

$$\begin{cases} \tilde{\mathbf{x}}_l = \text{RMSNorm}(\mathbf{x}_l) \\ \mathcal{H}_l^{\text{pre}} = \alpha_l^{\text{pre}} \cdot \tanh(\theta_l^{\text{pre}} \tilde{\mathbf{x}}_l^\top) + \mathbf{b}_l^{\text{pre}} \\ \mathcal{H}_l^{\text{post}} = \alpha_l^{\text{post}} \cdot \tanh(\theta_l^{\text{post}} \tilde{\mathbf{x}}_l^\top) + \mathbf{b}_l^{\text{post}} \\ \mathcal{H}_l^{\text{res}} = \alpha_l^{\text{res}} \cdot \tanh(\theta_l^{\text{res}} \tilde{\mathbf{x}}_l^\top) + \mathbf{b}_l^{\text{res}}, \end{cases} \quad (5)$$

where  $\text{RMSNorm}(\cdot)$  (Zhang & Sennrich, 2019) is applied to the last dimension, and the scalars  $\alpha_l^{\text{pre}}, \alpha_l^{\text{post}}$  and  $\alpha_l^{\text{res}} \in$

**Table 1. Ablation Study of HC Components.** When a specific mapping ( $\mathcal{H}_l^{\text{pre}}$ ,  $\mathcal{H}_l^{\text{post}}$ , or  $\mathcal{H}_l^{\text{res}}$ ) is disabled, we employ a fixed mapping to maintain dimensional consistency: one-hot weights for  $\mathcal{H}_l^{\text{pre}}$ , uniform weights of ones for  $\mathcal{H}_l^{\text{post}}$ , and the identity matrix for  $\mathcal{H}_l^{\text{res}}$ .

| $\mathcal{H}_l^{\text{res}}$ | $\mathcal{H}_l^{\text{pre}}$ | $\mathcal{H}_l^{\text{post}}$ | Absolute Loss Gap ( $\downarrow$ ) |
|------------------------------|------------------------------|-------------------------------|------------------------------------|
|                              |                              |                               | 0.0                                |
| ✓                            |                              |                               | − 0.022                            |
|                              | ✓                            |                               | − 0.016                            |
|                              |                              | ✓                             | − 0.025                            |
| ✓                            | ✓                            | ✓                             | − 0.027                            |

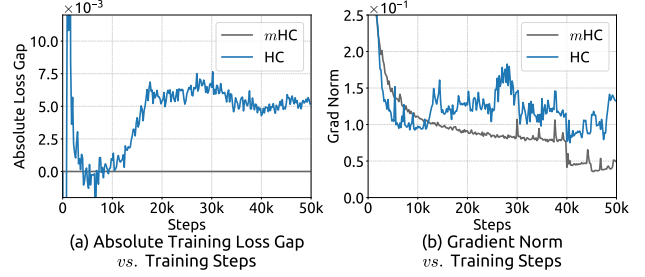
$\mathbb{R}$  are learnable gating factors initialized to small values. The dynamic mappings are derived via linear projections parameterized by  $\theta_l^{\text{pre}}, \theta_l^{\text{post}} \in \mathbb{R}^{1 \times C}$  and  $\theta_l^{\text{res}} \in \mathbb{R}^{n \times C}$ , while the static mappings are represented by learnable biases  $\mathbf{b}_l^{\text{pre}}, \mathbf{b}_l^{\text{post}} \in \mathbb{R}^{1 \times n}$  and  $\mathbf{b}_l^{\text{res}} \in \mathbb{R}^{n \times n}$ .

It is worth noting that the introduction of these mappings— $\mathcal{H}_l^{\text{pre}}$ ,  $\mathcal{H}_l^{\text{post}}$ , and  $\mathcal{H}_l^{\text{res}}$ —incurs negligible computational overhead, as the typical expansion rate  $n$ , e.g. 4, is much smaller than the input dimension  $C$ . With this design, HC effectively decouples the information capacity of the residual stream from the layer’s input dimension, which is strongly correlated with the model’s computational complexity (FLOPs). Consequently, HC offers a new avenue for scaling by adjusting the residual stream width, complementing the traditional scaling dimensions of model FLOPs and training data size discussed in pre-training scaling laws (Hoffmann et al., 2022).

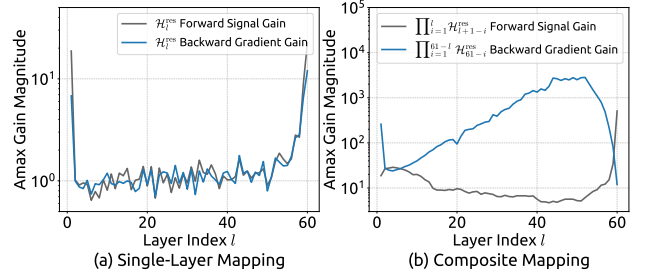
Although HC necessitates three mappings to manage the dimensional mismatch between the residual stream and the layer input, preliminary experiments presented in Tab. 1 indicate that the residual mapping  $\mathcal{H}_l^{\text{res}}$  yields the most significant performance gain and the setting without  $\mathcal{H}_l^{\text{res}}$  suffers from more gradient norm spikes. This finding underscores the critical importance of effective information exchange within the residual stream.

### 3.1. Numerical Instability

While the residual mapping  $\mathcal{H}_l^{\text{res}}$  is instrumental for performance, its sequential application poses a significant risk to numerical stability. As detailed in Eq. (4), when HC is extended across multiple layers, the effective signal propagation from layer  $l$  to  $L$  is governed by the composite mapping  $\prod_{i=1}^{L-l} \mathcal{H}_{L-i}^{\text{res}}$ . Since the learnable mapping  $\mathcal{H}_l^{\text{res}}$  is unconstrained, this composite mapping inevitably deviates from the identity mapping. Consequently, the signal magnitude is prone to explosion or vanishing during both the forward pass and backpropagation. This phenomenon undermines the fundamental premise of residual learning, which relies on unimpeded signal flow, thereby destabilizing the training



**Figure 2. Training Instability of Hyper-Connections (HC).** This figure illustrates (a) the absolute loss gap of HC relative to mHC, and (b) the comparison of gradient norms. All results are based on 27B models.



**Figure 3. Propagation Instability of Hyper-Connections (HC).** This figure illustrates the propagation dynamics of (a) the single-layer mapping  $\mathcal{H}_l^{\text{res}}$  and (b) the composite mapping  $\prod_{i=1}^{L-l} \mathcal{H}_{L-i}^{\text{res}}$  within the 27B model. The layer index  $l$  (x-axis) unrolls each standard Transformer block into two independent layers (Attention and FFN). The Amax Gain Magnitude (y-axis) is calculated as the maximum absolute row sum (for the forward signal) and column sum (for the backward gradient), averaged over all tokens in a selected sequence.

process in deeper or larger-scale models.

Empirical evidence supports this analysis. We observe unstable loss behavior in large-scale experiments, as illustrated in Fig. 2. Taking mHC as the baseline, HC exhibits an unexpected loss surge around the 12k step, which is highly correlated with the instability in the gradient norm. Furthermore, the analysis on  $\mathcal{H}_l^{\text{res}}$  validates the mechanism of this instability. To quantify how the composite mapping  $\prod_{i=1}^{L-l} \mathcal{H}_{L-i}^{\text{res}}$  amplifies signals along the residual stream, we utilize two metrics. The first, based on the maximum absolute value of the row sums of the composite mapping, captures the worst-case expansion in the forward pass. The second, based on the maximum absolute column sum, corresponds to the backward pass. We refer to these metrics as the *Amax Gain Magnitude* of the composite mapping. As shown in Fig. 3 (b), the Amax Gain Magnitude yields extreme values with peaks of 3000, a stark divergence from 1 that confirms the presence of exploding residual streams.

### 3.2. System Overhead

While the computational complexity of HC remains manageable due to the linearity of the additional mappings, the

system-level overhead presents a non-negligible challenge. Specifically, memory access (I/O) costs often constitute one of the primary bottlenecks in modern model architectures. This bottleneck is frequently overlooked in architectural design, yet it decisively impacts runtime efficiency (Dao et al., 2022).

In the HC formulation, memory access overhead related to the residual stream increases by a factor proportional to  $n$ . This excessive I/O demand significantly degrades training throughput without the mitigation of fused kernels. Besides, since  $\mathcal{H}_l^{\text{pre}}$ ,  $\mathcal{H}_l^{\text{post}}$ , and  $\mathcal{H}_l^{\text{res}}$  involve learnable parameters, their intermediate activations are required for backpropagation. This results in a substantial increase in the GPU memory footprint, often necessitating gradient checkpointing to maintain feasible memory usage. Furthermore, HC requires  $n$ -fold more communication cost in pipeline parallelism (Qi et al., 2024), leading to larger bubbles and decreasing the training throughput.

## 4. Method

### 4.1. Manifold-Constrained Hyper-Connections

Drawing inspiration from the identity mapping principle (He et al., 2016b), the core premise of mHC is to constrain the residual mapping  $\mathcal{H}_l^{\text{res}}$  onto a specific manifold. While the original identity mapping ensures stability by enforcing  $\mathcal{H}_l^{\text{res}} = \mathbf{I}$ , it fundamentally precludes information exchange within the residual stream, which is critical for maximizing the potential of multi-stream architectures. Therefore, we propose projecting the residual mapping onto a manifold that simultaneously maintains the stability of signal propagation across layers and facilitates mutual interaction among residual streams to preserve the model’s expressivity. To this end, we restrict  $\mathcal{H}_l^{\text{res}}$  to be a doubly stochastic matrix, which has non-negative entries where both the rows and columns sum to 1. Formally, let  $\mathcal{M}^{\text{res}}$  denote the manifold of doubly stochastic matrices (also known as the Birkhoff polytope). We constrain  $\mathcal{H}_l^{\text{res}}$  to  $\mathcal{P}_{\mathcal{M}^{\text{res}}}(\mathcal{H}_l^{\text{res}})$ , defined as:

$$\mathcal{P}_{\mathcal{M}^{\text{res}}}(\mathcal{H}_l^{\text{res}}) := \left\{ \mathcal{H}_l^{\text{res}} \in \mathbb{R}^{n \times n} \mid \mathcal{H}_l^{\text{res}} \mathbf{1}_n = \mathbf{1}_n, \right. \\ \left. \mathbf{1}_n^\top \mathcal{H}_l^{\text{res}} = \mathbf{1}_n^\top, \mathcal{H}_l^{\text{res}} > 0 \right\}, \quad (6)$$

where  $\mathbf{1}_n$  represents the  $n$ -dimensional vector of all ones. We exclude the boundary of the Birkhoff polytope to ensure it constitutes a smooth manifold.

It is worth noting that when  $n = 1$ , the doubly stochastic condition degenerates to the scalar 1, thereby recovering the original identity mapping. The choice of double stochasticity confers several rigorous theoretical properties beneficial for large-scale model training:

1. **Norm Preservation:** The spectral norm of a doubly

stochastic matrix is bounded by 1 (i.e.,  $\|\mathcal{H}_l^{\text{res}}\|_2 \leq 1$ ). This implies that the learnable mapping is non-expansive, effectively mitigating the gradient explosion problem.

2. **Compositional Closure:** The set of doubly stochastic matrices is closed under matrix multiplication. This ensures that the composite residual mapping across multiple layers,  $\prod_{i=1}^{L-l} \mathcal{H}_{L-i}^{\text{res}}$ , remains doubly stochastic, thereby preserving stability throughout the entire depth of the model.
3. **Geometric Interpretation via the Birkhoff Polytope:** The set  $\mathcal{M}^{\text{res}}$  forms the Birkhoff polytope, which is the convex hull of the set of permutation matrices. This provides a clear geometric interpretation: the residual mapping acts as a convex combination of permutations. Mathematically, the repeated application of such matrices tends to increase the mixing of information across streams monotonically, effectively functioning as a robust feature fusion mechanism.

Additionally, we impose non-negativity constraints on the input mappings  $\mathcal{H}_l^{\text{pre}}$  and output mappings  $\mathcal{H}_l^{\text{post}}$ . This constraint prevents signal cancellation arising from the composition of positive and negative coefficients, which can also be considered as a special manifold projection.

### 4.2. Parameterization and Manifold Projection

In this section, we detail the calculation process of  $\mathcal{H}_l^{\text{pre}}$ ,  $\mathcal{H}_l^{\text{post}}$ , and  $\mathcal{H}_l^{\text{res}}$  in mHC. Given the input hidden matrix  $\mathbf{x}_l \in \mathbb{R}^{n \times C}$  at the  $l$ -th layer, we first flatten it into a vector  $\tilde{\mathbf{x}}_l = \text{vec}(\mathbf{x}_l) \in \mathbb{R}^{1 \times nC}$  to preserve full context information. Then, we follow the original HC formulation to get the dynamic mappings and the static mappings as follows:

$$\begin{cases} \tilde{\mathbf{x}}'_l = \text{RMSNorm}(\tilde{\mathbf{x}}_l) \\ \tilde{\mathcal{H}}_l^{\text{pre}} = \alpha_l^{\text{pre}} \cdot (\tilde{\mathbf{x}}'_l \phi_l^{\text{pre}}) + \mathbf{b}_l^{\text{pre}} \\ \tilde{\mathcal{H}}_l^{\text{post}} = \alpha_l^{\text{post}} \cdot (\tilde{\mathbf{x}}'_l \phi_l^{\text{post}}) + \mathbf{b}_l^{\text{post}} \\ \tilde{\mathcal{H}}_l^{\text{res}} = \alpha_l^{\text{res}} \cdot \text{mat}(\tilde{\mathbf{x}}'_l \phi_l^{\text{res}}) + \mathbf{b}_l^{\text{res}}, \end{cases} \quad (7)$$

where  $\phi_l^{\text{pre}}, \phi_l^{\text{post}} \in \mathbb{R}^{nC \times n}$  and  $\phi_l^{\text{res}} \in \mathbb{R}^{nC \times n^2}$  are linear projections for dynamic mappings and  $\text{mat}(\cdot)$  is a reshape function from  $\mathbb{R}^{1 \times n^2}$  to  $\mathbb{R}^{n \times n}$ .

Then, the final constrained mappings are obtained via:

$$\begin{cases} \mathcal{H}_l^{\text{pre}} = \sigma(\tilde{\mathcal{H}}_l^{\text{pre}}) \\ \mathcal{H}_l^{\text{post}} = 2\sigma(\tilde{\mathcal{H}}_l^{\text{post}}) \\ \mathcal{H}_l^{\text{res}} = \text{Sinkhorn-Knopp}(\tilde{\mathcal{H}}_l^{\text{res}}), \end{cases} \quad (8)$$

where  $\sigma(\cdot)$  denotes the Sigmoid function. The Sinkhorn-Knopp( $\cdot$ ) operator first makes all elements positive via exponentiation and then iteratively normalizes rows

and columns to sum to 1. Specifically, given a positive matrix  $\mathbf{M}^{(0)} = \exp(\tilde{\mathcal{H}}_l^{\text{res}})$  as the starting point, the normalization iteration proceeds as:

$$\mathbf{M}^{(t)} = \mathcal{T}_r \left( \mathcal{T}_c(\mathbf{M}^{(t-1)}) \right), \quad (9)$$

where  $\mathcal{T}_r$  and  $\mathcal{T}_c$  denote row and column normalization, respectively. This process converges to a doubly stochastic matrix  $\mathcal{H}_l^{\text{res}} = \mathbf{M}^{(t_{\max})}$  as  $t_{\max} \rightarrow \infty$ . We choose  $t_{\max} = 20$  as a practical value in our experiments.

### 4.3. Efficient Infrastructure Design

In this section, we detail the infrastructure design tailored for *mHC*. Through rigorous optimization, we implement *mHC* (with  $n = 4$ ) in large-scale models with a marginal training overhead of only 6.7%.

#### 4.3.1. KERNEL FUSION

To mitigate the overhead of high-dimensional operations in *mHC*, we implement a series of aggressive kernel fusion strategies. First, observing that RMSNorm imposes significant latency on the high-dimensional hidden state  $\tilde{\mathbf{x}}_l \in \mathbb{R}^{1 \times nC}$ , we reorder the dividing-by-norm operation to follow the matrix multiplication. This optimization maintains mathematical equivalence while improving efficiency.

Furthermore, we fuse multiple operations with common memory access into unified compute kernels to reduce memory bandwidth bottlenecks. For the application of coefficients, we introduce specialized kernels for  $\mathcal{F}_{\text{pre}} := \mathcal{H}_l^{\text{pre}} \mathbf{x}_l$  and  $\mathcal{F}_{\text{post, res}} := \mathcal{H}_l^{\text{res}} \mathbf{x}_l + \mathcal{H}_l^{\text{post}^\top} \mathcal{F}(\cdot, \cdot)$ . By fusing the application of  $\mathcal{H}_l^{\text{post}}$  and  $\mathcal{H}_l^{\text{res}}$  with residual merging, we reduce memory reads and writes to  $\frac{1}{3}$ .

The majority of these kernels are implemented using TileLang (Wang et al., 2025), which allows us to maximize memory bandwidth utilization with minimal engineering effort. Detailed mathematical formulations and specific kernel fusion strategies are provided in Appendix A.1.

#### 4.3.2. RECOMPUTING

The  $n$ -stream residual design introduces substantial memory overhead during training. To mitigate this, we discard the intermediate activations of the *mHC* kernels after the forward pass and recompute them on-the-fly in the backward pass, through re-executing the *mHC* kernels without the heavy layer function  $\mathcal{F}$ . Consequently, for a block of  $L_r$  consecutive layers, we need only store the input  $\mathbf{x}_{l_0}$  to the first layer. Excluding lightweight coefficients while accounting for the pre-norm within  $\mathcal{F}$ , Tab. 2 summarizes the intermediate activations preserved for the backward pass.

Since recomputation of *mHC* kernels is performed for blocks of  $L_r$  consecutive layers, given a total of  $L$  lay-

ers, we persistently store the first layer input  $\mathbf{x}_{l_0}$  of all  $\lceil \frac{L}{L_r} \rceil$  blocks for the backward pass. In addition to this resident memory, the recomputation process introduces a transient memory overhead of  $(n+2)C \times L_r$  elements for the active block, which determines the peak memory usage during backpropagation. Consequently, we determine the optimal block size  $L_r^*$  by minimizing the total memory footprint corresponding to  $L_r$ :

$$L_r^* = \arg \min_{L_r} \left[ nC \times \left\lceil \frac{L}{L_r} \right\rceil + (n+2)C \times L_r \right] \approx \sqrt{\frac{nL}{n+2}}. \quad (10)$$

Furthermore, pipeline parallelism in large-scale training imposes a constraint: recomputation blocks must not cross pipeline stage boundaries. Observing that the theoretical optimum  $L_r^*$  typically aligns with the number of layers per pipeline stage, we choose to synchronize the recomputation boundaries with the pipeline stages.

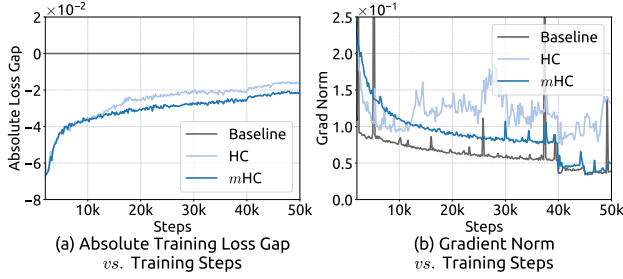
#### 4.3.3. OVERLAPPING COMMUNICATION IN DUALPIPE

In large-scale training, pipeline parallelism is the standard practice for mitigating parameter and gradient memory footprints. Specifically, we adopt the DualPipe schedule (Liu et al., 2024b), which effectively overlaps scale-out interconnected communication traffic, such as those in expert and pipeline parallelism. However, compared to the single-stream design, the proposed  $n$ -stream residual in *mHC* incurs substantial communication latency across pipeline stages. Furthermore, at stage boundaries, the recomputation of *mHC* kernels for all  $L_r$  layers introduces non-negligible computational overhead. To address these bottlenecks, we extend the DualPipe schedule (see Appendix A.2) to facilitate improved overlapping of communication and computation at pipeline stage boundaries.

Notably, to prevent blocking the communication stream, we execute the  $\mathcal{F}_{\text{post, res}}$  kernels of MLP layers on a dedicated high-priority compute stream. We further refrain from employing persistent kernels for long-running operations in attention layers, thereby preventing extended stalls. This design enables the preemption of overlapped attention computations, allowing for flexible scheduling while main-

**Table 2. Stored and Recomputed Intermediate Activations.** We list per token activation preserved for the backward pass and the transient activation recomputed in  $L_r$  consecutive layers. Layer  $l_0$  represents the first layer in  $L_r$  layers and layer  $l$  is in  $[l_0, l_0 + L_r - 1]$ .

| Activations                                                           | Size | Stored Method                 |
|-----------------------------------------------------------------------|------|-------------------------------|
| $\mathbf{x}_{l_0}$                                                    | $nC$ | Every $L_r$ layers            |
| $\mathcal{F}(\mathcal{H}_l^{\text{pre}} \mathbf{x}_l, \mathcal{W}_l)$ | $C$  | Every layer                   |
| $\mathbf{x}_l$                                                        | $nC$ | Transient inside $L_r$ layers |
| $\mathcal{H}_l^{\text{pre}} \mathbf{x}_l$                             | $C$  | Transient inside $L_r$ layers |
| $\text{RMSNorm}(\mathcal{H}_l^{\text{pre}} \mathbf{x}_l)$             | $C$  | Transient inside $L_r$ layers |



**Figure 4. Training Stability of Manifold-Constrained Hyper-Connections (*m*HC).** This figure illustrates (a) the absolute training loss gap of *m*HC and HC relative to the residual connection (i.e. baseline), and (b) the gradient norm of the three methods. All experiments utilize the 27B model. The results demonstrate that *m*HC exhibits improved stability in terms of both loss and gradient norm.

taining high utilization of the compute device’s processing units. Furthermore, the recomputation process is decoupled from pipeline communication dependencies, as the initial activation of each stage  $\mathbf{x}_{l_0}$  is already cached locally.

## 5. Experiments

### 5.1. Experimental Setup

We validate the proposed method via language model pre-training. Utilizing MoE architectures inspired by DeepSeek-V3 (Liu et al., 2024b), we adopt its key architectural designs at reduced scales and train four distinct model variants to cover different evaluation regimes. Based on these model variants, we conduct a comparative analysis between the residual connection (i.e. baseline), HC, and our proposed *m*HC. Specifically, the expansion rate  $n$  for both HC and *m*HC is set to 4. Our primary focus is a 27B model trained with a dataset size proportional to its parameters, which serves as the subject for our system-level main results. Expanding on this, we analyze the compute scaling behavior by incorporating smaller 3B and 9B models trained with proportional data, which allows us to observe performance trends across varying compute. Additionally, to specifically investigate the token scaling behavior, we train a separate 3B model on a fixed corpus of 1 trillion tokens. Detailed model configurations and training hyper-parameters are provided in Appendix A.5. Furthermore, the initialization of *m*HC follows the principle established in HC, which ensures that the initial state of *m*HC closely resembles the standard residual connection. To achieve this, we initialize all linear projections for the dynamic mappings to zero and carefully select the initial values of the static mappings such that at the beginning of training,  $\mathcal{H}^{\text{res}}$  closely approximates the identity matrix,  $\mathcal{H}^{\text{pre}}$  is close to a one-hot vector and  $\mathcal{H}^{\text{post}}$  is an all-ones vector. The complete initialization details and pseudo implementation are provided in Appendix A.6.

**Table 3. System-level Benchmark Results for 27B Models.** This table compares the zero-shot and few-shot performance of the residual connection (i.e. baseline), HC, and *m*HC across 8 diverse downstream benchmarks. *m*HC consistently outperforms the Baseline and surpasses HC on the majority of benchmarks, demonstrating its effectiveness in large-scale pre-training.

| Benchmark (Metric) | BBH (EM)    | DROP (F1)   | GSM8K (EM)  | HellaSwag (Acc.) | MATH (EM)   | MMLU (Acc.) | PIQA (Acc.) | TriviaQA (EM) |
|--------------------|-------------|-------------|-------------|------------------|-------------|-------------|-------------|---------------|
| # Shots            | 3           | 1           | 8           | 10               | 4           | 5           | 0           | 5             |
| Baseline           | 43.8        | 47.0        | 46.7        | 73.7             | 22.0        | 59.0        | 78.5        | 54.3          |
| w/ HC              | 48.9        | 51.6        | 53.2        | 74.3             | <b>26.4</b> | 63.0        | 79.9        | 56.3          |
| w/ <i>m</i> HC     | <b>51.0</b> | <b>53.9</b> | <b>53.8</b> | <b>74.7</b>      | 26.0        | <b>63.4</b> | <b>80.5</b> | <b>57.6</b>   |

### 5.2. Main Results

We begin by examining the training stability and convergence of the 27B models. As illustrated in Fig. 4 (a), *m*HC effectively mitigates the training instability observed in HC, achieving a final loss reduction of 0.021 compared to the baseline. This improved stability is further corroborated by the gradient norm analysis in Fig. 4 (b), where *m*HC exhibits significantly better behavior than HC, maintaining a stable profile comparable to the baseline.

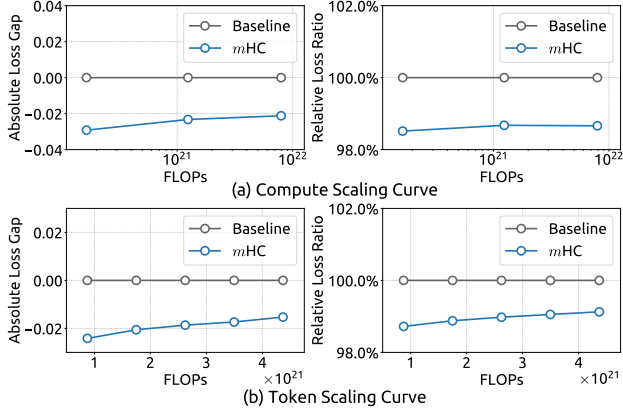
Tab. 3 presents the downstream performance across a diverse set of benchmarks (Hendrycks et al., 2020; Cobbe et al., 2021; Zellers et al., 2019; Hendrycks et al., 2021; Bisk et al., 2020; Joshi et al., 2017). *m*HC yields comprehensive improvements, consistently outperforming the baseline and HC on the majority of tasks. Notably, compared to HC, *m*HC further enhances the model’s reasoning capabilities, delivering performance gains of 2.1% on BBH (Suzgun et al., 2023) and 2.3% on DROP (Dua et al., 2019).

### 5.3. Scaling Experiments

To assess the scalability of our approach, we report the relative loss improvement of *m*HC against the baseline across different scales. In Fig. 5 (a), we plot the compute scaling curve spanning 3B, 9B, and 27B parameters. The trajectory indicates that the performance advantage is robustly maintained even at higher computational budgets, showing only marginal attenuation. Furthermore, we examine the within-run dynamics in Fig. 5 (b), which presents the token scaling curve for the 3B model. Collectively, these findings validate the effectiveness of *m*HC in large-scale scenarios.

### 5.4. Stability Analysis

Similar to Fig. 3, Fig. 6 illustrates the propagation stability of *m*HC. Ideally, the single-layer mapping satisfies the doubly stochastic constraint, implying that both the forward signal gain and the backward gradient gain should equal 1.



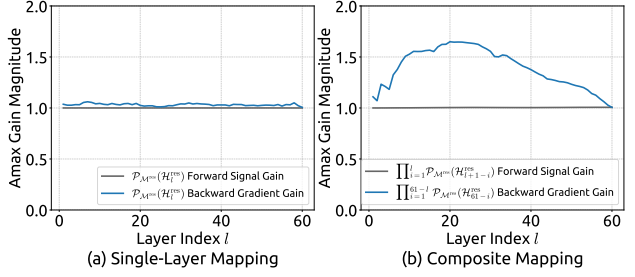
**Figure 5. Scaling Properties of mHC Compared to the Residual Connection (i.e. Baseline).** (a) **Compute Scaling Curve.** Solid lines depict the performance gap across different compute budgets. Each point represents a specific compute-optimal configuration of model size and dataset size, scaling from 3B and 9B to 27B parameters. (b) **Token Scaling Curve.** Trajectory of the 3B model during training. Each point represents the model’s performance at different training tokens. Detailed architectures and training configurations are provided in Appendix A.5.

However, practical implementations utilizing the Sinkhorn-Knopp algorithm must limit the number of iterations to achieve computational efficiency. In our settings, we use 20 iterations to obtain an approximate solution. Consequently, as shown in Fig. 6 (a), the backward gradient gain deviates slightly from 1. In the composite case shown in Fig. 6 (b), the deviation increases but remains bounded, reaching a maximum value of approximately 1.6. Notably, compared to the maximum gain magnitude of nearly 3000 in HC, mHC significantly reduces it by three orders of magnitude. We also visualize the maximum Amax Gain Magnitude of the composite residual mapping in Fig. 7. The maximum gain steadily increases during HC training, but remains stable in the mHC model. One potential explanation is that in the absence of constraints, the unconstrained streams progressively amplify their individual contributions in a self-reinforcing manner. These results demonstrate that mHC significantly enhances propagation stability compared to HC, ensuring stable forward signal and backward gradient flows. Additionally, we display representative mappings in Appendix A.3. We observe that for HC, when the maximum gain is large, other values also tend to be significant, which indicates general instability across all propagation paths. In contrast, mHC consistently yields stable results.

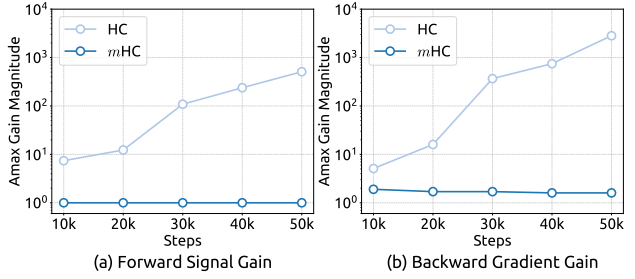
### 5.5. Ablation Studies

We conduct ablation studies on a 3B parameter MoE model to validate specific design choices in mHC.

**Omission of tanh Activation** The original HC formulation employs a tanh activation to bound the dynamic coefficients within the range  $[-1, 1]$ . However, mHC already



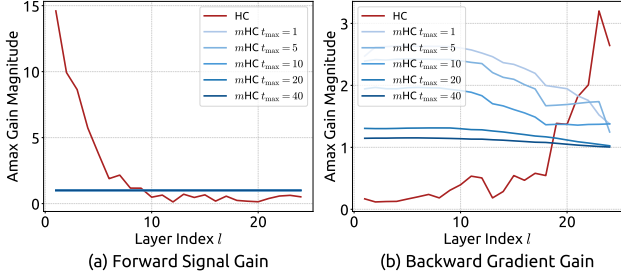
**Figure 6. Propagation Stability of Manifold-Constrained Hyper-Connections (mHC) across Layers.** This figure illustrates the propagation dynamics of (a) the single-layer mapping  $\mathcal{P}_{\mathcal{M}^{\text{res}}}(\mathcal{H}_l^{\text{res}})$  and (b) the composite mapping  $\prod_{i=1}^l \mathcal{P}_{\mathcal{M}^{\text{res}}}(\mathcal{H}_{l+1-i}^{\text{res}})$  and  $\prod_{i=1}^{l-1} \mathcal{P}_{\mathcal{M}^{\text{res}}}(\mathcal{H}_{l-i}^{\text{res}})$  within the 27B model. The results demonstrate that mHC significantly enhances propagation stability compared to HC.



**Figure 7. Propagation Stability of Manifold-Constrained Hyper-Connections (mHC) across Training Steps.** This figure illustrates the (a) forward and (b) backward composite propagation dynamics over all layers  $\prod_{i=1}^{L-1} \mathcal{P}_{\mathcal{M}^{\text{res}}}(\mathcal{H}_{L-i}^{\text{res}})$  of the 27B model across training steps. We report the worst-case composite mapping gain. The results demonstrate that mHC stabilizes signal propagation throughout training.

enforces structural constraints on the residual mapping. Consequently, the element-wise constraint of tanh is redundant and unnecessarily limits the capability of the mappings. As evidenced in Tab. 4, reintroducing tanh results in a performance degradation, increasing the loss by 0.005.

**Sinkhorn-Knopp Iterations  $t_{\text{max}}$**  The hyperparameter  $t_{\text{max}}$  in Eq. (9) dictates the number of renormalization steps in the Sinkhorn-Knopp algorithm. Tab. 4 indicates that different iteration steps yield similar performance. We further analyze the convergence speed of the Sinkhorn-Knopp algorithm by evaluating the Amax Gain Magnitude of the 3B model under varying  $t_{\text{max}}$ . Fig. 8 shows that the stability improvement is already substantial at  $t_{\text{max}} = 1$ , with increasing  $t_{\text{max}}$  yielding steady but diminishing returns. Crucially, this iteration insensitivity strengthens our core argument: performance gains stem from the manifold constraint mechanism pulling the matrix toward the Birkhoff polytope, rather than requiring perfectly exact doubly stochastic projections. Therefore, we adopt  $t_{\text{max}} = 20$  as the default to maintain competitive performance while guaranteeing robust mani-



**Figure 8. Convergence Speed of the Sinkhorn-Knopp Algorithm.** This figure illustrates the (a) forward and (b) backward composite propagation dynamics  $\prod \mathcal{P}_{\mathcal{M}^{\text{res}}}(\mathcal{H}^{\text{res}})$  of the 3B model with varying numbers of Sinkhorn-Knopp iterations. The results show that even a single iteration drastically alters the properties of  $\mathcal{H}^{\text{res}}$ , significantly reducing both forward and backward Amax Gain Magnitudes.

fold constraints.

**Input Dimension for Dynamic Projection** Unlike the original HC formulation, which processes each stream independently, *mHC* concatenates the  $n$ -stream inputs into a unified vector  $\bar{\mathbf{x}}_l \in \mathbb{R}^{1 \times nC}$  for coefficient calculation (as shown in Eq. (7)). This design enables dynamic mappings to leverage complete context information across all streams, while reverting to the 1-stream setting leads to a 0.004 increase in loss as shown in Tab. 4. Although this design increases the FLOPs of the projection, the system overhead remains manageable because these kernels are primarily memory-bound when  $n$  is small.

## 6. Conclusion and Outlook

In this paper, we identify that while expanding the width of residual stream and diversifying connections yields performance gains as proposed in Hyper-Connections (HC), the unconstrained nature of these connections leads to signal divergence. This disruption compromises the conservation of signal energy across layers, inducing training instability and hindering the scalability of deep networks. To address these challenges, we introduce **Manifold-Constrained Hyper-Connections (*mHC*)**, a generalized framework that projects the residual connection space onto a specific manifold. By employing the Sinkhorn-Knopp algorithm to enforce a doubly stochastic constraint on residual mappings, *mHC* transforms signal propagation into a convex combination of features. Empirical results confirm that *mHC* effectively restores the identity mapping property, enabling stable large-scale training with superior scalability compared to conventional HC. Crucially, through efficient infrastructure-level optimizations, *mHC* delivers these improvements with negligible computational overhead.

As a generalized extension of the HC paradigm, *mHC* opens several promising avenues for future research. Although this

**Table 4. Ablation Study of *mHC* Design Choices.**

| Keep tanh | 1-stream projection | $t_{\text{max}}$ value | Absolute Loss Gap ( $\downarrow$ ) |
|-----------|---------------------|------------------------|------------------------------------|
| ✓         | ✓                   | 20                     | 0.0                                |
|           |                     | 20                     | + 0.005                            |
|           |                     | 20                     | + 0.004                            |
|           |                     | 10                     | 0.0                                |
|           |                     | 40                     | 0.0                                |

work utilizes doubly stochastic matrices to ensure stability, the framework accommodates the exploration of diverse manifold constraints tailored to specific learning objectives. We anticipate that further investigation into distinct geometric constraints could yield novel methods that better optimize the trade-off between plasticity and stability. Furthermore, we hope *mHC* rejuvenates community interest in macro-architecture design. By deepening the understanding of how topological structures influence optimization and representation learning, *mHC* will help address current limitations and potentially illuminate new pathways for the evolution of next-generation foundational architectures.

## Impact Statement

This paper presents work whose goal is to advance the field of machine learning. There are many potential societal consequences of our work, none of which we feel must be specifically highlighted here.

## References

- Ainslie, J., Lee-Thorp, J., De Jong, M., Zemlyanskiy, Y., Lebrón, F., and Sanghai, S. Gqa: Training generalized multi-query transformer models from multi-head checkpoints. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 4895–4901, 2023.
- Bisk, Y., Zellers, R., Gao, J., Choi, Y., et al. Piqa: Reasoning about physical commonsense in natural language. In *Proceedings of the AAAI conference on artificial intelligence*, volume 34, pp. 7432–7439, 2020.
- Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33: 1877–1901, 2020.
- Chai, Y., Jin, S., and Hou, X. Highway transformer: Self-gating enhanced self-attentive networks. In Jurafsky, D., Chai, J., Schluter, N., and Tetreault, J. (eds.), *Proceedings of the 58th Annual Meeting of the Associa-*

- tion for Computational Linguistics, pp. 6887–6900, Online, July 2020. Association for Computational Linguistics. doi: 10.18653/v1/2020.acl-main.616. URL <https://aclanthology.org/2020.acl-main.616/>.
- Chollet, F. Xception: Deep learning with depthwise separable convolutions. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 1251–1258, 2017.
- Cobbe, K., Kosaraju, V., Bavarian, M., Chen, M., Jun, H., Kaiser, L., Plappert, M., Tworek, J., Hilton, J., Nakano, R., et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- Dao, T., Fu, D. Y., Ermon, S., Rudra, A., and Ré, C. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- Dua, D., Wang, Y., Dasigi, P., Stanovsky, G., Singh, S., and Gardner, M. DROP: A reading comprehension benchmark requiring discrete reasoning over paragraphs. In Burstein, J., Doran, C., and Solorio, T. (eds.), *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT 2019, Minneapolis, MN, USA, June 2-7, 2019, Volume 1 (Long and Short Papers)*, pp. 2368–2378. Association for Computational Linguistics, 2019. doi: 10.18653/V1/N19-1246. URL <https://doi.org/10.18653/v1/n19-1246>.
- Fang, Y., CAI, Y., Chen, J., Zhao, J., Tian, G., and Li, G. Cross-layer retrospective retrieving via layer attention. In *The Eleventh International Conference on Learning Representations*, 2023. URL <https://openreview.net/forum?id=pvgELlyS3Ql>.
- Fedus, W., Zoph, B., and Shazeer, N. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022.
- He, K., Zhang, X., Ren, S., and Sun, J. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 770–778, 2016a.
- He, K., Zhang, X., Ren, S., and Sun, J. Identity mappings in deep residual networks. In *European conference on computer vision*, pp. 630–645. Springer, 2016b.
- Heddes, M., Javanmard, A., Axiotis, K., Fu, G., Bateni, M., and Mirrokni, V. Deepcrossattention: Supercharging transformer residual connections. In *Forty-second International Conference on Machine Learning*, 2025. URL <https://openreview.net/forum?id=j3JBfFnGYh>.
- Hendrycks, D., Burns, C., Basart, S., Zou, A., Mazeika, M., Song, D., and Steinhardt, J. Measuring massive multitask language understanding. *arXiv preprint arXiv:2009.03300*, 2020.
- Hendrycks, D., Burns, C., Kadavath, S., Arora, A., Basart, S., Tang, E., Song, D., and Steinhardt, J. Measuring mathematical problem solving with the math dataset. *arXiv preprint arXiv:2103.03874*, 2021.
- Hoffmann, J., Borgeaud, S., Mensch, A., Buchatskaya, E., Cai, T., Rutherford, E., de Las Casas, D., Hendricks, L. A., Welbl, J., Clark, A., Hennigan, T., Noland, E., Millican, K., van den Driessche, G., Damoc, B., Guy, A., Osindero, S., Simonyan, K., Elsen, E., Vinyals, O., Rae, J., and Sifre, L. An empirical analysis of compute-optimal large language model training. *Advances in neural information processing systems*, 35:30016–30030, 2022.
- Huang, G., Liu, Z., Van Der Maaten, L., and Weinberger, K. Q. Densely connected convolutional networks. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 4700–4708, 2017.
- Joshi, M., Choi, E., Weld, D., and Zettlemoyer, L. TriviaQA: A large scale distantly supervised challenge dataset for reading comprehension. In Barzilay, R. and Kan, M.-Y. (eds.), *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 1601–1611, Vancouver, Canada, July 2017. Association for Computational Linguistics. doi: 10.18653/v1/P17-1147. URL <https://aclanthology.org/P17-1147>.
- Kornblith, S., Norouzi, M., Lee, H., and Hinton, G. Similarity of neural network representations revisited. In *International conference on machine learning*, pp. 3519–3529. PMIR, 2019.
- Larsson, G., Maire, M., and Shakhnarovich, G. Fractalnet: Ultra-deep neural networks without residuals. *arXiv preprint arXiv:1605.07648*, 2016.
- Lepikhin, D., Lee, H., Xu, Y., Chen, D., Firat, O., Huang, Y., Krikun, M., Shazeer, N., and Chen, Z. Gshard: Scaling giant models with conditional computation and automatic sharding. *arXiv preprint arXiv:2006.16668*, 2020.
- Liu, A., Feng, B., Wang, B., Wang, B., Liu, B., Zhao, C., Deng, C., Ruan, C., Dai, D., Guo, D., et al. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model. *arXiv preprint arXiv:2405.04434*, 2024a.

- Liu, A., Feng, B., Xue, B., Wang, B., Wu, B., Lu, C., Zhao, C., Deng, C., Zhang, C., Ruan, C., et al. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437*, 2024b.
- Loshchilov, I. and Hutter, F. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101*, 2017.
- Mak, B. and Flanigan, J. Residual matrix transformers: Scaling the size of the residual stream. *arXiv preprint arXiv:2506.22696*, 2025.
- Menghani, G., Kumar, R., and Kumar, S. LAurel: Learned augmented residual layer. In *Forty-second International Conference on Machine Learning*, 2025. URL <https://openreview.net/forum?id=rUDRWP9WvZ>.
- Pagliardini, M., Mohtashami, A., Fleuret, F., and Jaggi, M. Denseformer: Enhancing information flow in transformers via depth weighted averaging. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024. URL <https://openreview.net/forum?id=kMnoh7CXrq>.
- Qi, P., Wan, X., Huang, G., and Lin, M. Zero bubble (almost) pipeline parallelism. In *The Twelfth International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=tuzTN0eIO5>.
- Shazeer, N. Fast transformer decoding: One write-head is all you need. *arXiv preprint arXiv:1911.02150*, 2019.
- Shazeer, N., Mirhoseini, A., Maziarz, K., Davis, A., Le, Q., Hinton, G., and Dean, J. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *arXiv preprint arXiv:1701.06538*, 2017.
- Sinkhorn, R. and Knopp, P. Concerning nonnegative matrices and doubly stochastic matrices. *Pacific Journal of Mathematics*, 21(2):343–348, 1967.
- Srivastava, R. K., Greff, K., and Schmidhuber, J. Training very deep networks. *Advances in neural information processing systems*, 28, 2015.
- Su, J., Lu, Y., Pan, S., Murtadha, A., Wen, B., and Liu, Y. Roformer: Enhanced transformer with rotary position embedding. *arXiv preprint arXiv:2104.09864*, 2021.
- Suzgun, M., Scales, N., Schärli, N., Gehrmann, S., Tay, Y., Chung, H. W., Chowdhery, A., Le, Q., Chi, E., Zhou, D., et al. Challenging big-bench tasks and whether chain-of-thought can solve them. In *Findings of the Association for Computational Linguistics: ACL 2023*, pp. 13003–13051, 2023.
- Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M.-A., Lacroix, T., Rozière, B., Goyal, N., Hambro, E., Azhar, F., et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
- Wang, L., Gao, H., Zhao, C., Sun, X., and Dai, D. Auxiliary-loss-free load balancing strategy for mixture-of-experts. *arXiv preprint arXiv:2408.15664*, 2024.
- Wang, L., Cheng, Y., Shi, Y., Tang, Z., Mo, Z., Xie, W., Ma, L., Xia, Y., Xue, J., Yang, F., et al. Tilelang: A composable tiled programming model for ai systems. *arXiv preprint arXiv:2504.17577*, 2025.
- Xiao, D., Meng, Q., Li, S., and Yuan, X. Muddformer: Breaking residual bottlenecks in transformers via multiway dynamic dense connections. *arXiv preprint arXiv:2502.12170*, 2025.
- Xie, S., Girshick, R., Dollár, P., Tu, Z., and He, K. Aggregated residual transformations for deep neural networks. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 1492–1500, 2017.
- Xie, S., Zhang, H., Guo, J., Tan, X., Bian, J., Awadalla, H. H., Menezes, A., Qin, T., and Yan, R. Residual: Transformer with dual residual connections. *arXiv preprint arXiv:2304.14802*, 2023.
- Yu, F., Wang, D., Shelhamer, E., and Darrell, T. Deep layer aggregation. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 2403–2412, 2018.
- Zellers, R., Holtzman, A., Bisk, Y., Farhadi, A., and Choi, Y. HellaSwag: Can a machine really finish your sentence? In Korhonen, A., Traum, D. R., and Màrquez, L. (eds.), *Proceedings of the 57th Conference of the Association for Computational Linguistics, ACL 2019, Florence, Italy, July 28- August 2, 2019, Volume 1: Long Papers*, pp. 4791–4800. Association for Computational Linguistics, 2019. doi: 10.18653/v1/p19-1472. URL <https://doi.org/10.18653/v1/p19-1472>.
- Zhang, B. and Sennrich, R. Root mean square layer normalization. *Advances in neural information processing systems*, 32, 2019.
- Zhu, D., Huang, H., Huang, Z., Zeng, Y., Mao, Y., Wu, B., Min, Q., and Zhou, X. Hyper-connections. In *International Conference on Learning Representations*, volume 2025, pp. 97183–97219, 2025.

## A. Appendix

### A.1. Kernel Fusion Details

In this section, we provide the detailed mathematical framework and implementation specifics for the fused kernels used in *mHC*. We employ mixed-precision strategies to maximize numerical accuracy without compromising training speed. Based on the inputs and parameters detailed in Eq. (11) to (14), we implement three specialized kernels to compute  $\mathcal{H}_l^{\text{pre}}$ ,  $\mathcal{H}_l^{\text{post}}$ , and  $\mathcal{H}_l^{\text{res}}$ . In these kernels, the biases and linear projections are consolidated into  $\mathbf{b}_l$  and  $\phi_l$ , and the RMSNorm weight is also absorbed in  $\phi_l$ .

$$\phi_l : \text{tfloat32} \quad [nC, n^2 + 2n] \quad (11)$$

$$\vec{\mathbf{x}}_l : \text{bfloat16} \quad [1, nC] \quad (12)$$

$$\alpha_l^{\text{pre}}, \alpha_l^{\text{post}}, \alpha_l^{\text{res}} : \text{float32} \quad \text{Scalars} \quad (13)$$

$$\mathbf{b}_l : \text{float32} \quad [1, n^2 + 2n] \quad (14)$$

$$[\tilde{\mathcal{H}}_l^{\text{pre}}, \tilde{\mathcal{H}}_l^{\text{post}}, \tilde{\mathcal{H}}_l^{\text{res}}] : \text{float32} \quad = \vec{\mathbf{x}}_l \phi_l \quad (15)$$

$$r : \text{float32} \quad = \|\vec{\mathbf{x}}_l\|_2 / \sqrt{nC} \quad (16)$$

$$[\tilde{\mathcal{H}}_l^{\text{pre}}, \tilde{\mathcal{H}}_l^{\text{post}}, \tilde{\mathcal{H}}_l^{\text{res}}] : \text{float32} \quad = 1/r \left[ \alpha_l^{\text{pre}} \tilde{\mathcal{H}}_l^{\text{pre}}, \alpha_l^{\text{post}} \tilde{\mathcal{H}}_l^{\text{post}}, \alpha_l^{\text{res}} \tilde{\mathcal{H}}_l^{\text{res}} \right] + \mathbf{b}_l \quad (17)$$

$$\mathcal{H}_l^{\text{pre}} : \text{float32} \quad = \sigma \left( \tilde{\mathcal{H}}_l^{\text{pre}} \right) \quad (18)$$

$$\mathcal{H}_l^{\text{post}} : \text{float32} \quad = 2\sigma \left( \tilde{\mathcal{H}}_l^{\text{post}} \right) \quad (19)$$

$$\mathcal{H}_l^{\text{res}} : \text{float32} \quad = \text{Sinkhorn-Knopp} \left( \tilde{\mathcal{H}}_l^{\text{res}} \right) \quad (20)$$

We categorize the optimization into three primary fusion strategies:

- **Projection and Normalization Fusion** (Eq. 15–16): We develop a unified kernel that fuses two scans on  $\vec{\mathbf{x}}_l$ , leveraging matrix multiplication units to maximize memory bandwidth utilization. The backward pass—comprising two matrix multiplications—is similarly consolidated into a single kernel, eliminating redundant reloading of  $\vec{\mathbf{x}}_l$ . Both kernels feature a finely tuned pipeline (load, cast, compute, store) to handle mixed-precision processing efficiently.
- **Coefficient Mapping Fusion** (Eq. 17–19): These lightweight operations on small coefficients are fused into a single kernel, significantly reducing kernel launch overhead.
- **Sinkhorn-Knopp Iteration** (Eq. 20): We implement the Sinkhorn-Knopp iteration within a single kernel. For the backward pass, we derive a custom backward kernel that recomputes intermediate results on-chip and traverses the entire iteration.

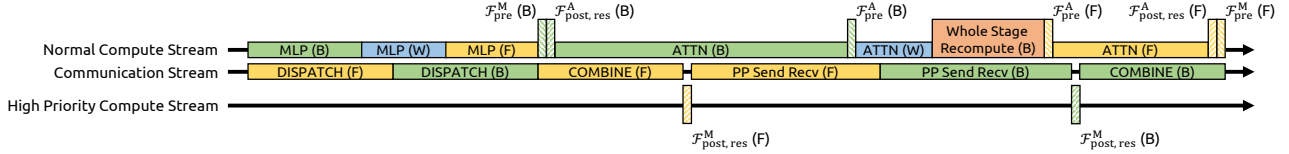
Notably, while these operations are maintained as distinct fused groups to facilitate gradient computation during training, they can be further fused into a single monolithic kernel during the **inference phase** to achieve even higher execution efficiency and reduced latency.

Focusing on the widely adopted pre-norm Transformer (Vaswani et al., 2017) architecture, we analyze the I/O patterns inherent to HC and *mHC*. Tab. 5 summarizes the per token memory access overhead in a single residual layer forward introduced by the *n*-stream residual design. The analysis reveals that this design increases the memory access cost by a factor approximately proportional to *n*. By leveraging kernel fusion, *mHC* reduces memory reads by a factor of 2 and memory writes by a factor of 4 during the training forward pass.

**Table 5. Comparison of Memory Access Costs Per Token.** Values represent element counts. This analysis accounts for residual stream maintenance overhead, excluding internal  $\mathcal{F}$  I/O. In common practice, since  $C$  is significantly larger than  $n$ , only the element counts associated with  $C$  are presented here.

| Operation                       | Read                          | Write                         |
|---------------------------------|-------------------------------|-------------------------------|
| <b>Residual Connection</b>      |                               |                               |
| Residual Merge                  | $2C$                          | $C$                           |
| <b>Total I/O</b>                | <b><math>2C</math></b>        | <b><math>C</math></b>         |
| <b>Hyper-Connections</b>        |                               |                               |
| Calculate $\mathcal{H}_l^*$     | $2nC$                         | $nC$                          |
| $\mathcal{H}_l^{\text{pre}}$    | $nC$                          | $C$                           |
| $\mathcal{H}_l^{\text{post}}$   | $C$                           | $nC$                          |
| $\mathcal{H}_l^{\text{res}}$    | $nC$                          | $nC$                          |
| Residual Merge                  | $2nC$                         | $nC$                          |
| <b>Total I/O</b>                | <b><math>(6n + 1)C</math></b> | <b><math>(4n + 1)C</math></b> |
| <b>mHC</b>                      |                               |                               |
| Calculate $\mathcal{H}_l^*$     | $nC$                          | -                             |
| $\mathcal{F}_{\text{pre}}$      | $nC$                          | $C$                           |
| $\mathcal{F}_{\text{post,res}}$ | $(n + 1)C$                    | $nC$                          |
| <b>Total I/O</b>                | <b><math>(3n + 1)C</math></b> | <b><math>(n + 1)C</math></b>  |

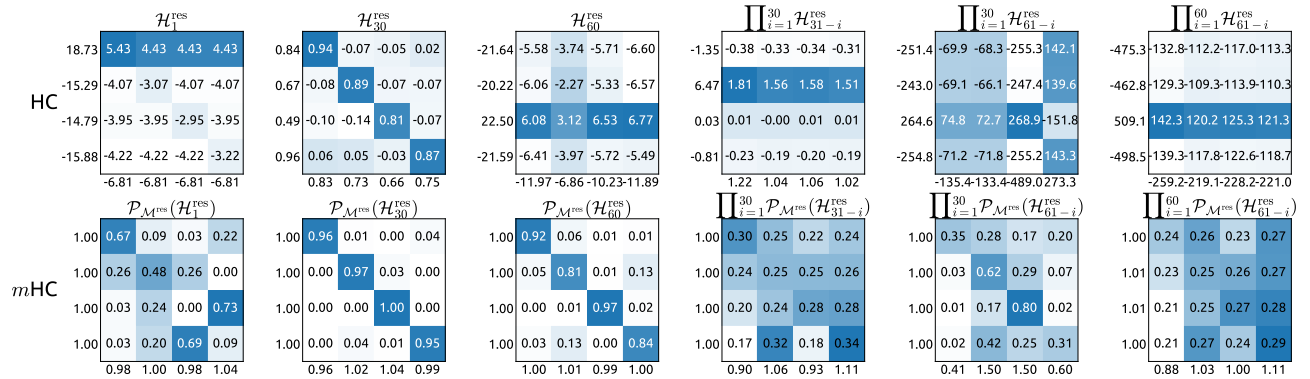
## A.2. Illustration of Overlapping Communication in DualPipe



**Figure 9. Communication-Computation Overlapping for mHC.** We extend the DualPipe schedule to handle the overhead introduced by mHC. Lengths of each block are illustrative only and do not represent actual duration. (F), (B), (W) refer to forward pass, backward pass, weight gradient computation, respectively.  $\mathcal{F}^A$  and  $\mathcal{F}^M$  represent kernels corresponding to Attention and MLP, respectively.

## A.3. Visualizations

### A.3.1. VISUALIZATIONS OF LEARNABLE MAPPINGS



**Figure 10. Visualizations of Learnable Mappings.** This figure displays representative single-layer and composite mappings for HC (first row) and mHC (second row). Each matrix is computed by averaging over all tokens within a selected sequence. The labels annotated along the y-axis and x-axis indicate the forward signal gain (row sum) and the backward gradient gain (column sum), respectively.

Fig. 10 illustrates the learnable residual mappings of the 27B *mHC* model.

We further analyze the composite product of residual mappings within consecutive layer groups in the 27B *mHC* model. Tab. 6 reports the normalized squared Frobenius norm of these composite matrices, defined as  $\frac{\|\prod \mathcal{P}_{\mathcal{M}^{\text{res}}}(\mathcal{H}_i^{\text{res}})\|_F^2 - 1}{n-1}$ , where a value of 0 corresponds to the flat doubly stochastic matrix  $(1/n)\mathbf{1}\mathbf{1}^\top$  and 1 corresponds to a permutation matrix. The results show that the composite mappings in shallow layers (Layer 1–10) are closer to uniform mixing, while those in the middle and deeper layers maintain substantially higher heterogeneity. This indicates that both forward features and backward gradients undergo non-trivial local transformations, demonstrating that the model actively exchanges and aggregates information across multiple streams throughout training.

Table 6. Composite Mappings Analysis between Adjacent Layers

|                           | Layer<br>1-10 | Layer<br>11-20 | Layer<br>21-30 | Layer<br>31-40 | Layer<br>41-50 | Layer<br>51-60 |
|---------------------------|---------------|----------------|----------------|----------------|----------------|----------------|
| Normalized Squared F-Norm | 0.09          | 0.49           | 0.70           | 0.62           | 0.71           | 0.36           |

### A.3.2. VISUALIZATIONS OF *n*-STREAM RESIDUALS

We further analyze the geometric characteristics of different residual streams. Specifically, we compare the RMS norms, pairwise cosine similarities, and Centered Kernel Alignment (CKA) similarities (Kornblith et al., 2019) across the four streams of the 3B *mHC* model. Fig. 11 shows the complete visualization. The results indicate significant differences between streams, particularly in their norms and CKA similarities, suggesting that each stream captures distinct representational features. This observation highlights the rich geometric structure enabled by multi-stream architectures, which we consider a promising direction for future investigation.

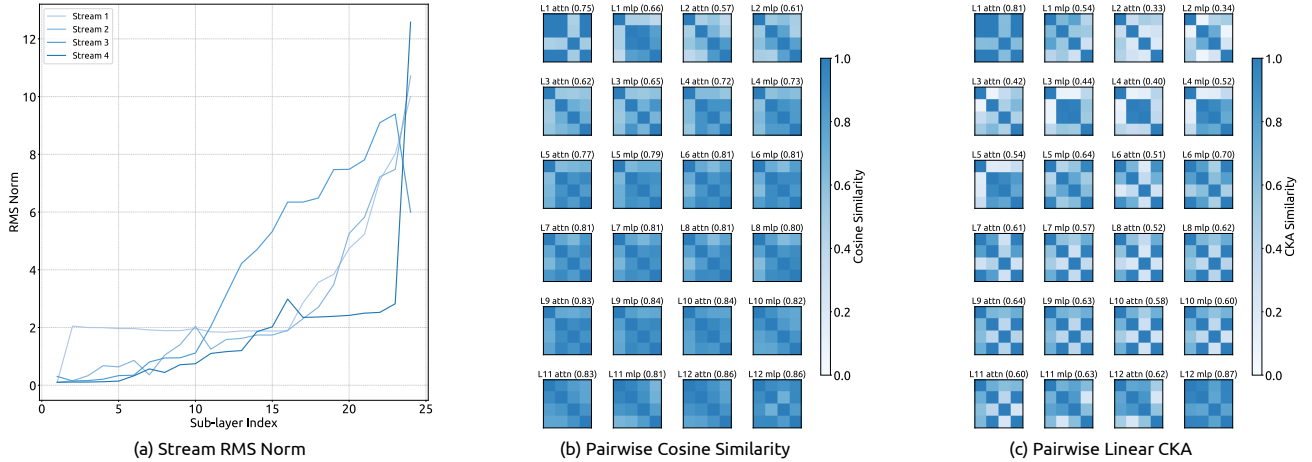


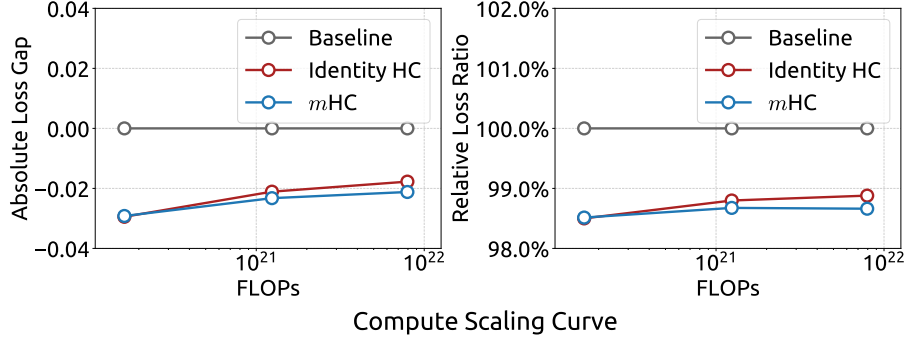
Figure 11. **Visualizations of *n*-Stream Residuals.** We illustrate the norms, cosine similarities, and Centered Kernel Alignment (CKA) similarities (Kornblith et al., 2019) across different streams of the 3B *mHC* model. The mean cosine and CKA similarities (excluding the diagonal) are noted on top of each subplot.

### A.4. Identity HC Experiments

In Tab. 1, we show that the residual mapping  $\mathcal{H}_l^{\text{res}}$  plays the most critical role in multi-stream residual architectures, and that the setting without  $\mathcal{H}_l^{\text{res}}$ , named *Identity HC*, suffers from more frequent gradient norm spikes. Here, we re-examine Identity HC within the *mHC* framework across multiple parameter scales by fixing  $\mathcal{H}^{\text{res}}$  to the identity matrix  $I$ . The results are shown in Fig. 12.

First, directly applying  $\mathcal{H}^{\text{res}} = I$  to the original HC architecture causes performance degradation and severe training collapse, likely due to conflicts with the  $\mathcal{H}^{\text{pre}}$  and  $\mathcal{H}^{\text{post}}$  mappings. In contrast, Identity HC within the *mHC* framework remains stable, demonstrating that its manifold constraints on  $\mathcal{H}^{\text{pre}}$  and  $\mathcal{H}^{\text{post}}$  mappings are highly robust and universally

effective. Second, the Identity HC performs comparably to *m*HC at 3B parameters. However, as the model scales to 9B parameters, *m*HC begins to slightly outperform the identity variant. Furthermore, for 27B models, *m*HC further outperforms the Identity HC. This widening performance gap suggests that *m*HC’s representational advantages will yield even greater improvements at larger scales. We hypothesize that *m*HC’s near-identity initialization allows it to learn complex, non-trivial doubly stochastic structures. While smaller models struggle to fully exploit this higher-order capacity, these benefits are unlocked as the parameter count increases.



*Figure 12. Compute Scaling Properties of *m*HC Compared to Identity HC.* Solid lines depict the performance gap across different compute budgets. Each point represents a specific compute-optimal configuration of model size and dataset size, scaling from 3B and 9B to 27B parameters. The results show that Identity HC has an inferior scaling property.

### A.5. Detailed Model Specifications and Hyper-parameters

**Table 7. Detailed Model Specifications and Hyper-parameters.** This table presents the architectural configurations for the 3B, 9B, and 27B models based on the DeepSeek-V3 (Liu et al., 2024b) architecture. It outlines the specific hyper-parameters for *m*HC and HC, including the residual stream expansion and Sinkhorn-Knopp settings, alongside the optimization and training protocols used in the experiments.

| Attribute                                  | 3B                            | 9B                                | 27B    | 3B<br>1T Tokens                |
|--------------------------------------------|-------------------------------|-----------------------------------|--------|--------------------------------|
| Vocab Params                               | 331M                          | 496M                              | 662M   | 331M                           |
| Active Params                              | 612M                          | 1.66B                             | 4.14B  | 612M                           |
| Total Params                               | 2.97B                         | 9.18B                             | 27.0B  | 2.97B                          |
| Layers                                     | 12                            | 18                                | 30     | 12                             |
| Leading Dense Layers                       |                               | 1                                 |        | 1                              |
| Routed Experts                             | 64                            | 64                                | 72     | 64                             |
| Active Experts                             |                               | 6                                 |        | 6                              |
| Shared Experts                             |                               | 2                                 |        | 2                              |
| Dimension                                  | 1280                          | 1920                              | 2560   | 1280                           |
| FFN Dimension                              | 896                           | 1280                              | 1536   | 896                            |
| Load Balancing Method                      | Loss-Free (Wang et al., 2024) |                                   |        | Loss-Free                      |
| Attention Heads                            | 16                            | 24                                | 32     | 16                             |
| Attention Dimension                        |                               | 128                               |        | 128                            |
| Attention Variant                          |                               | MLA (Liu et al., 2024a)           |        | MLA                            |
| KV Rank                                    |                               | 512                               |        | 512                            |
| Position Embedding                         |                               | RoPE (Su et al., 2021)            |        | RoPE                           |
| RoPE Dimension                             |                               | 64                                |        | 64                             |
| RoPE $\theta$                              |                               | 10000                             |        | 10000                          |
| Layer Norm Type                            |                               | RMSNorm (Zhang & Sennrich, 2019)  |        | RMSNorm                        |
| Layer Norm $\epsilon$                      |                               | 1e-20                             |        | 1e-20                          |
| <i>m</i> HC/HC Expansion Rate $n$          |                               | 4                                 |        | 4                              |
| <i>m</i> HC/HC Gating Factor Init $\alpha$ |                               | 0.01                              |        | 0.01                           |
| <i>m</i> HC Sinkhorn-Knopp $t_{\max}$      |                               | 20                                |        | 20                             |
| Sequence Length                            |                               | 4096                              |        | 4096                           |
| Vocab Size                                 |                               | 129280                            |        | 129280                         |
| Batch Size                                 | 320                           | 512                               | 1280   | 2560                           |
| Training Steps                             | 30000                         | 50000                             | 50000  | 100000                         |
| Training Tokens                            | 39.3B                         | 105B                              | 262B   | 1.05T                          |
| Warmup Steps                               |                               | 2000                              |        | 2000                           |
| Optimizer                                  |                               | AdamW (Loshchilov & Hutter, 2017) |        | AdamW                          |
| AdamW Betas                                |                               | (0.9, 0.95)                       |        | (0.9, 0.95)                    |
| AdamW $\epsilon$                           |                               | 1e-20                             |        | 1e-20                          |
| Base Learning Rate                         | 8.6e-4                        | 5.9e-4                            | 4.0e-4 | 9.0e-4                         |
| Lr Scheduler                               |                               | Step                              |        | Step                           |
| Lr Decay Step Ratio                        |                               | [0.8 $\times$ , 0.9 $\times$ ]    |        | [0.8 $\times$ , 0.9 $\times$ ] |
| Lr Decay Rate                              |                               | [0.316, 0.1]                      |        | [0.316, 0.1]                   |
| Weight Decay                               |                               | 0.1                               |        | 0.1                            |

## A.6. Pseudo Implementation of *mHC*

---

**Algorithm 1** Pseudo code of *mHC* in PyTorch-like style.

---

```

class mHC(torch.nn.Module):
    def __init__(self, dim, nstreams, layer_id, block_func):
        super(mHC, self).__init__()

        self.dim = dim
        self.nstreams = nstreams
        self.layer_id = layer_id
        self.block_func = block_func

        self.alpha = torch.nn.Parameter(torch.ones(3,) * 0.01)
        self.bias = torch.nn.Parameter(
            torch.cat([
                -3.0 * torch.ones(self.nstreams),
                torch.zeros(self.nstreams),
                (
                    6.0 * torch.eye(self.nstreams, self.nstreams) - 3.0
                ).flatten(),
            ]),
        )
        self.bias.data[self.layer_id % self.nstreams] = 3.0
        self.weight = torch.nn.Parameter(
            torch.zeros((2 + self.nstreams) * self.nstreams, self.nstreams * self.dim)
        )
        self.norm = torch.nn.RMSNorm(self.nstreams * self.dim, eps=1e-20)
        self.mhc_eps = 1e-6

    def sinkhorn(self, H, repeat_times=20):
        H = H.softmax(-1) + self.mhc_eps
        H = H / (H.sum(-2, keepdim=True) + self.mhc_eps)
        for _ in range(repeat_times - 1):
            H = H / (H.sum(-1, keepdim=True) + self.mhc_eps)
            H = H / (H.sum(-2, keepdim=True) + self.mhc_eps)
        return H

    def forward(self, x):
        # x: Input (BxLxNxNxC)
        x_flatten = x.flatten(2)
        x_norm = self.norm(x_flatten)

        # H: Coefficient Before Manifold-Constrained (BxLx(N^2+2N))
        repeats = torch.tensor([self.nstreams, self.nstreams, self.nstreams**2], device=x.device)
        alpha_expanded = torch.repeat_interleave(self.alpha, repeats)
        H = x_norm @ self.weight.T * alpha_expanded[None, None, :] + self.bias[None, None, :]

        # Coefficient With Manifold-Constrained
        H_pre = torch.sigmoid(H[:, :, :self.nstreams]) + self.mhc_eps
        H_post = torch.sigmoid(H[:, :, self.nstreams:self.nstreams*2]) * 2.0
        H_res = self.sinkhorn(H[:, :, self.nstreams*2:].view(x.size(0), x.size(1), self.nstreams, self.nstreams))

        x_input = torch.einsum('bln,blnc->blc', H_pre, x)
        y = self.block_func(x_input)

        return torch.einsum('bln,blc->blnc', H_post, y) + torch.einsum('blmn,blmc->blnc', H_res, x)

```

---